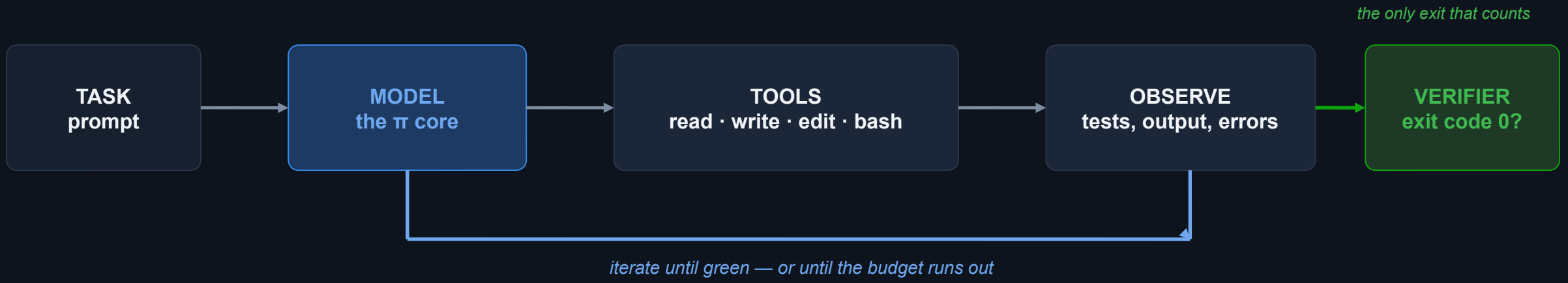
THE BIG IDEA

Models aren't chatbots here — they're loop cores

An agent loop: the model reads a task, acts through tools, observes reality, and iterates until an independent judge says done.



file-path discipline

artifacts must land where the grader looks

recover from red

read a pytest traceback, make the smallest fix

drive multi-step tools

no wandering, no giving up early

know when it's done

stop on green — don't 'improve' passing code

The scientific discovery cycle

Implementation synthesis based on DOE’s closed-loop vision

01 Frame the question

Set the scientific objective, constraints and success criteria.

02 Assemble evidence

Retrieve relevant observations, theory and prior experiments.

03 Propose and predict

Generate hypotheses and estimate which tests will teach the most.

04 Run the test

Execute approved simulations or physical experiments.

05 Validate the result

Compare prediction with measurement and quantify uncertainty.

06 Update and repeat

Preserve the evidence and use it to choose the next question.

Synthesis informed by DOE National Challenges and Models teams

A portfolio of scientific models

Different scientific tasks need different model capabilities.

Reasoning models

Interpret questions, assemble evidence and plan scientific work.

Domain foundation models

Represent scientific structures and relationships in specialized data.

Surrogates and predictors

Approximate costly calculations and estimate promising candidates.

Evaluation and model registry

Track capabilities, limitations, versions and evidence for reuse.

Open-weight scientific models add a path for community adaptation and scrutiny.

A discovery requires models to compose

Question: which oxygen-containing alloy should we make next, and why?

AtomFM + OperatorFM
Predict candidate energetics and mechanical response

➡

ProcessFM + ControlFM
Choose a feasible melt and informative test

➡

InstrumentFM
Infer structure and defects from microscopy

⬅

Measured structure and properties update the next prediction

CodeFM executes simulations. EvidenceFM records the chain of justification.

Syllabus

CMSC 35200 Deep Learning Systems
Syllabus Outline

- Framing (Foundation)
- GPT implementation
- Self-Attention/Alternatives
- Large-scale Training/Parallelism
- Hardware Architectures for DL
- Foundation Models beyond LLM
- Multimodal Models
- Supporting Long-Context
- Pipeline/Finetuning
- Model Evaluation
- Agents and Ensembles
- Inference and Software Stack
- AI applications Frameworks
- Safety-Risks-Ethics
- End-to-End Problem Solving
- Hypothesis Formation
- AGI?
- Superintelligence?

Grades, Class, Projects, etc.

- **We will have ~5 labs**
  - We will be setting up access to Enpoints, agents, etc.
  - Labs/Problem Sets are generally done individually unless we specify otherwise
  - We have access to several LLMs captively running on our machines

- **We will have group projects that has a poster presentation and paper**
  - ~3 people per project
  - Project proposals due in a few weeks (we give feedback)
  - Projects result in Poster at end of Quarter and Paper

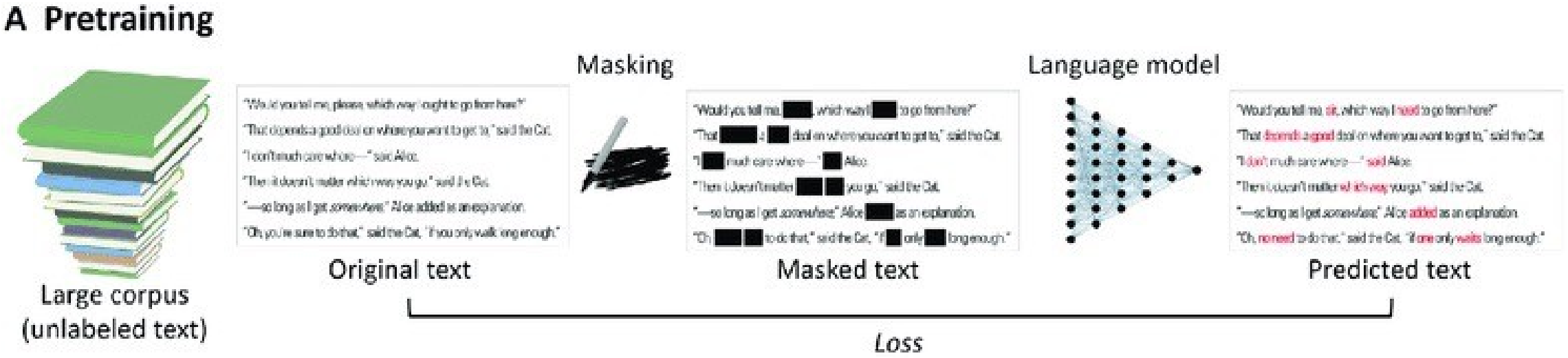
  (in the past some papers have resulted in publications!)

- **Grades**
  - Labs/Problem Sets 50% of the grade
  - Project 40% of the grade
  - Class engagement and participation 10%

Papers and background reading/watching

- There is no text for this course as it would be instantly out of date and useless
- On Slack/GitHub/Google Docs we will post papers for you to read and pointers to lectures and videos to watch to get needed background for our topics
- This class is not a general class on machine learning or deep learning so we either assume you already have exposure or you can do sufficient background reading and exploring to be comfortable with the basics
- You will need to know python and agents and be fairly agile with using systems, planning and running computational experiments and understanding what your code and the system is doing
- We want you to use LLMs as much as possible to help you learn material

Self-Supervised Pretraining and Fine-Tuning



Key Transformer Paper

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Study Guides and using LLMs

- I have used “Deep Research” , to generate comprehensive study guides for the class topics

Advanced Study Guide_ Language Model...es, Tokenization, and Evaluation Metrics.pdf	Aug 18, 2025 at 5:10 PM	95 KB	PDF
Advanced Study Guide_ Transformer Architecture and Evolution (2017–2022).pdf	Aug 18, 2025 at 5:10 PM	163 KB	PDF
Data Curation at Scale_ Quality, Filtering, Deduplication, and Documentation.pdf	Aug 18, 2025 at 5:10 PM	172 KB	PDF
Deep Research Report superintelligence.pdf	Aug 18, 2025 at 5:10 PM	438 KB	PDF
Efficient Transformer Design for Long Contexts and Fast Inference.pdf	Aug 18, 2025 at 5:10 PM	276 KB	PDF
Emerging AI-Hardware Accelerators.pdf	Aug 18, 2025 at 5:10 PM	123 KB	PDF
Evaluation & Validation_ Breadth, Reliability & Significance.pdf	Aug 18, 2025 at 5:10 PM	196 KB	PDF
External Tools and Retrieval for Language Models_ RAG, REALM, RETRO, Toolformer.pdf	Aug 18, 2025 at 5:10 PM	581 KB	PDF
Governance, Safety, and Interpretability for AGI-Class Systems.pdf	Aug 18, 2025 at 5:10 PM	248 KB	PDF
Inference & Deployment_ Throughput, Latency, and Cost.pdf	Aug 18, 2025 at 5:10 PM	77 KB	PDF
Mathematical Foundations You’ll Actually Use.pdf	Aug 18, 2025 at 5:10 PM	163 KB	PDF
Optimization and Systems for Large-Scale Pre-Training.pdf	Aug 18, 2025 at 5:10 PM	85 KB	PDF
Parallelism for Training & Inference.pdf	Aug 18, 2025 at 5:10 PM	161 KB	PDF
Post-Training I – Instruction Tuning & RLHF (Algorithms & Math).pdf	Aug 18, 2025 at 5:10 PM	269 KB	PDF
Reasoning in LLMs_ Prompting, Test-Time Compute, and Process Supervision.pdf	Aug 18, 2025 at 5:10 PM	605 KB	PDF
Scaling Strategies & Forecasting Capability Frontiers.pdf	Aug 18, 2025 at 5:10 PM	364 KB	PDF

MODEL SERVER TESTING - ITERATION 1

llama70

rbdgx2.cels.anl.gov | http://103.101.203.226:/v1
Model: llama70
Status: Success
Started: 10:56:21 | Time: 0.28s
Response: OK Tokens: 52

```
10:56:21 Tokens: 52 (prompt=50, completion=2)
10:56:21 Response: OK (content received)
10:56:21 Response received in 0.25s
10:56:21 Sending request...
10:56:21 Creating OpenAI client...
10:56:21 Starting test...
10:56:21 API Base: http://103.101.203.226:/v1
```

oss120

rbh101.cels.anl.gov | http://66.55.67.65:80/v1
Model: oss120
Status: Success
Started: 10:56:21 | Time: 0.32s
Response: OK Tokens: 125

```
10:56:22 Tokens: 125 (prompt=86, completion=39)
10:56:21 Response: OK (content received)
10:56:21 Response received in 0.29s
10:56:21 Sending request...
10:56:21 Creating OpenAI client...
10:56:21 Starting test...
10:56:21 API Base: http://66.55.67.65:80/v1
```

qwen80

hcdgx2.cels.anl.gov | http://103.90.163.143:80/v1
Model: qwen80
Status: Success
Started: 10:56:21 | Time: 0.88s
Response: OK Tokens: 25

```
10:56:22 Tokens: 25 (prompt=23, completion=2)
10:56:22 Response: OK (content received)
10:56:22 Response received in 0.85s
10:56:21 Sending request...
10:56:21 Creating OpenAI client...
10:56:21 Starting test...
10:56:21 API Base: http://103.90.163.143:80/v1
```

oss20

uicgpu.cels.anl.gov | http://195.88.24.64:80/v1
Model: oss20
Status: Success
Started: 10:56:21 | Time: 0.64s
Response: OK Tokens: 136

```
10:56:22 Tokens: 136 (prompt=86, completion=50)
10:56:22 Response: OK (content received)
10:56:22 Response received in 0.61s
10:56:21 Sending request...
10:56:21 Creating OpenAI client...
10:56:21 Starting test...
10:56:21 API Base: http://195.88.24.64:80/v1
```

AIEN FLEET • SYSTEM MAP

Physical fleet, dual transport fabric, tunnels, execution paths, services, and Telaio observability

ARCHITECTURE 2026-09-13

CONTROL + OBSERVABILITY PLANE

dashboard.py
read-only status + navigation
• JSON / curses / problems
• freshness + intentional-down

Telaio discoveryd
parallel runtime probing
• TTL leases + failure streaks
• atomic latest.json snapshot

Rete
route + capability graph
• 28 services / 95 paths
• 98 execution paths

Mirabile + Piago
agent execution plane
• selftest / send / fanout
• Cesello + Flusso variants

FALDA
memory gateway + distiller
• gateway / tailnet
• distillation pipeline

UMP
durable memory plane
• Dropbox store + MCP
• index + spark backup

Sibline / NATS
sibling messaging
• broker :4222
• subscriber + heartbeat

Fleet state
current + proposed
• host-local cache today
• 5+2 replicated DB design

PHYSICAL FLEET

HOME / OPERATOR EDGE

Mac control nodes, three GB10 Sparks, small hosts

- **m1 / mini** — control hub • 100.86.220.115
- **CherryRd** — UI + operator • 100.87.221.1
- **spark-36ac** — GB10 • Ollama • UMP backup
- **spark-95fe** — GB10 • Ollama
- **spark-9611** — GB10 • Ollama • DwarfStar
- **nuc13 / nuc7** — small compute + GC
- **m1acbook family** — portable agents
- **m3acbook family** — Mirabile / Piago
- **xps • rpi-gateway • prokko** — edge / gateway / intentional-down inventory

CHICAGO RENDEZVOUS

Redundant public anchors and reverse-tunnel ingress

- **chicago-1** — 100.116.24.63 • ws/wss + reverse ports
- **chicago-2** — 100.72.226.1 • ws/wss + reverse ports
- **chicago-3** — 100.68.255.123 • ws/wss + reverse ports
- **chicago-4** — 100.70.79.43 • ws/wss + reverse ports
- **chicago-5** — 100.75.113.16 • ws/wss + reverse ports

Public model fronts: :80 / LiteLLM :41443

ARGONNE / CELS + JLSE

GPU inference, local agents, protected services

- **rbdgx1** — GPU • oss120
- **rbdgx2** — GPU • laguna-s
- **rbdgx3** — GPU • model services
- **rbh101** — H100 • model backend
- **hcdgx2** — GPU • inventory
- **studio-ts** — Argo bridge
- **chiatta00** — PVC • LiteLLM :8000
- **uicgpu** — UIC GPU • Ollama

Public model routes
Chicago reverse/autossh front doors
• oss120 • laguna • kimi • nemotron

ALCF + EXTERNAL SERVICES

MFA-gated leadership systems and cloud model providers

- **Aurora UAN** — SSH 2207 • cesh 3207
- **Polaris login** — SSH 2209 • cesh 3209
- **Crux UAN** — SSH 2218 • cesh 3218

ALCF inference
m1 wrappers + tunnels
• Sophia :8890
• Metis :8892
• Globus Compute routes

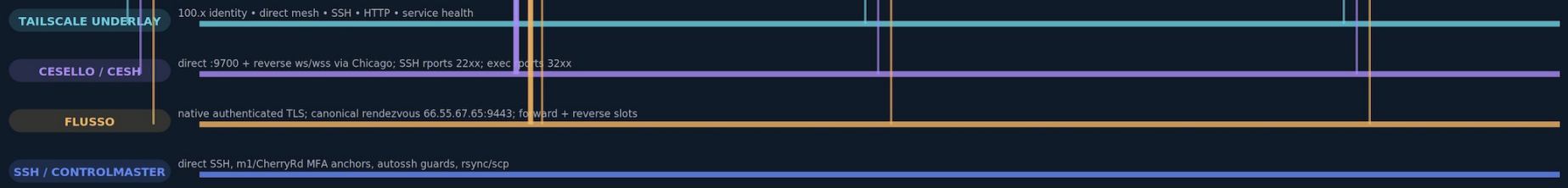
Cloud APIs
external provider plane
• OpenRouter
• Ollama Cloud Pro
• public LiteLLM

Operator UI
CherryRd-facing services
• Open WebUI :8081
• OpenClaw :18789
• Hermes gateway :8644

Model access layer
OpenAI-compatible endpoints + route redundancy
• LiteLLM aggregator :4000 + HA :4100 + public :41443
• Direct tailnet, CEZ proofs, autossh guards, VPS reverse proxies
• Spark Ollama :11434 • CELS backends • Chiatta aggregator :8000

CONNECTIVITY FABRIC

Shared rails summarize many-to-many attachment; detailed failover is held in Rete + Telaio state



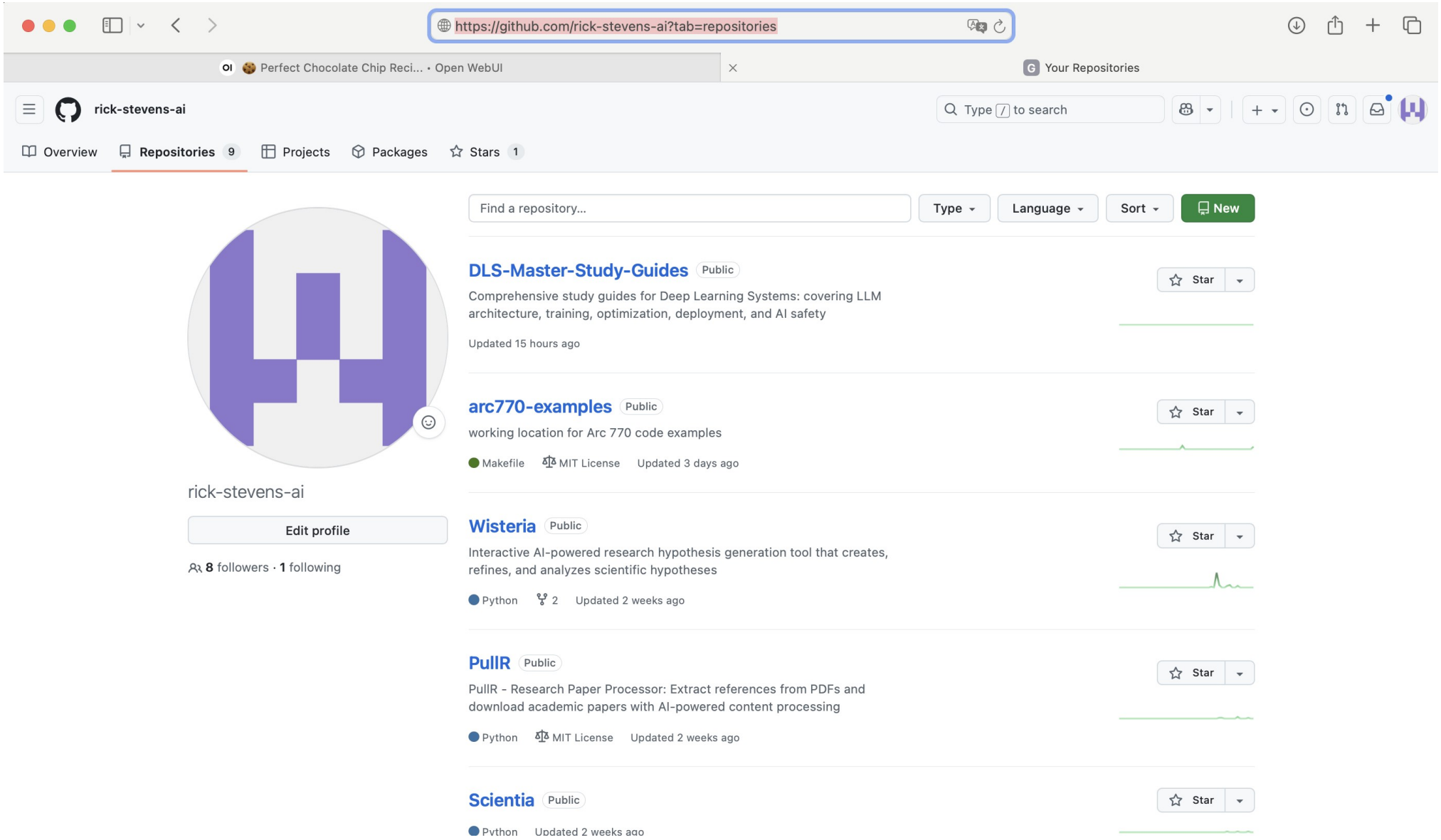
HOW THE SYSTEM WORKS

1. **Declare** — fleet_hosts.json; fleet-services.json; Rete graph
2. **Observe** — Telaio discoveryd; SSH / cesh / HTTP; TTL + failure streak
3. **Persist** — atomic latest.json; host-local dashboard cache; 5+2 DB design
4. **Interpret** — dashboard.py; intentional-down + freshness; route-specific diagnosis
5. **Operate** — Mirabile / Piago; CEZ + Flusso + SSH; agent fanout
6. **Serve** — LiteLLM + Ollama; FALDA + UMP; Sibline + APIs

LEGEND management · runtime/healthy · Cesello · Flusso · memory/data · external/MFA

Source: dashboard.py • Telaio discovery • Rete • fleet registry

https://github.com/rick-stevens-ai





rick-stevens-ai

Edit profile

17 followers · **1** following

Achievements



You unlocked new Achievements with private contributions! Show them off by including private contributions in your Profile in settings.

Popular repositories

Customize your pins

Argonium Public

HTML 6 5

DLS-Master-Study-Guides Public

Comprehensive study guides for Deep Learning Systems: covering LLM architecture, training, optimization, deployment, and AI safety

4

falda Public

FALDA — clustered hierarchical memory for scientific agents. Open, US-origin, self-hosted. Four tiers (Stream/Atoms/Scenes/Core) over SQLite + sqlite-vec + FTS5 with RRF-fused hybrid recall. Apache...

TypeScript 4 5

OpenCode-on-SPARK Public

"Run OpenCode with MiniMax-M2.1 on DGX Spark"

Shell 3 2

replication-project Public

use of AI and agents to reproduce the results in papers

HTML 3

ATPI Public

Shell 3 4

ChatGPT

Scaling Strategies & Forecasting Capability Frontiers

Introduction

Scaling up AI models has been a driving force behind recent breakthroughs in machine learning, especially with large language models (LLMs). **Scaling strategies** refer to how we increase model size, training data, and compute in order to improve performance. By studying empirical *scaling laws*, researchers have uncovered predictable relationships between these factors and model capability [1]. These insights enable **compute-optimal planning** – figuring out the best way to allocate a fixed compute budget across model parameters, number of training tokens, and training steps for maximum performance [2]. At the same time, as models approach higher capability, we face questions of **data quality** (are we running out of high-quality data to feed ever-larger models?), **reliability of reasoning** (do bigger models reason more robustly or do new failure modes emerge?), and ultimately **forecasting "AGI" timelines** (predicting when AI might reach human-level general intelligence). This study guide will cover these topics in detail, drawing on key papers: Kaplan *et al.* (2020) on early scaling laws, Hoffmann *et al.* (2022) on *compute-optimal* training (the *Chinchilla* study), Sagawa *et al.* (2023) on capability forecasting (scaling perspectives), and Bubeck *et al.* (2023) on emergent abilities in GPT-4. We will also discuss how to formulate a **group scaling-plan proposal** – effectively applying scaling laws in practice – in a case-study style. Throughout, we'll use real scaling curves and examples from the literature to illustrate concepts.

Compute-Optimal Scaling: Parameters ↔ Tokens ↔ Steps

Kaplan et al. (2020): Power-Law Scaling and Optimal Compute Allocation

In 2020, Kaplan *et al.* published a seminal work on *scaling laws for neural language models* [3]. They trained language models across **seven orders of magnitude** in scale and observed that the *test loss* (cross-entropy) follows a clear **power-law decline** as functions of three key factors: **model size (N)**, **dataset size (D)**, and **training compute (C)** [2] [1]. Figure 1 below (from Kaplan *et al.*) shows the core result: each plot is linear on a log-log scale, indicating a power-law relationship. Performance improves (loss decreases) smoothly as we increase compute, data, or model parameters, until eventually hitting an irreducible loss floor (due to the intrinsic entropy of natural language) [4] [5].

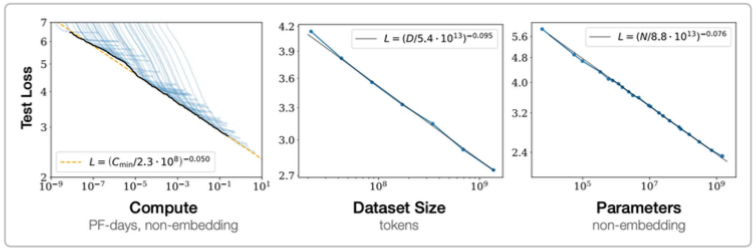


Figure 1: Scaling laws from Kaplan et al. (2020) [4] [5]. Test loss follows a power law as model parameters (right), dataset size in tokens (center), or compute in petaflop-days (left) increase. Doubling model size, for example, reduces loss by a constant factor (~5% reduction per doubling) [5].

A remarkable finding was that these trends held over vast scales without saturating (within the tested range). The authors summarized the relationships in simple power-law forms. For instance, they estimated:

- **Model size law:** $L(N) \propto N^{-0.076}$ – doubling the number of non-embedding parameters *N* reduces loss by about 5% [5].
- **Data size law:** $L(D) \propto D^{-0.095}$ – doubling the training tokens reduces loss ~6.4% [5].
- **Compute law:** $L(C) \propto C^{-0.050}$ – doubling total compute (FLOPs) yields ~3.4% less loss [5].

These exponents (0.076, 0.095, 0.050) quantify *diminishing returns*: more data helps more than more parameters (per doubling), and both help more than longer training on the same data. Importantly, Kaplan *et al.* also fit *multivariate* scaling laws to capture interactions. They proposed a combined loss function of model and data:

$$L(N, D) = \left[(N/N_c)^{\alpha_N} + (D/D_c)^{\alpha_D}\right]^{-1/\alpha_D},$$

with $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.103$ (and N_c, D_c constants) [6] [7]. This equation implies an **optimal trade-off**: given a fixed compute budget, there is a best way to split it between training a larger model vs. training on more data. Kaplan *et al.* used this to determine the **compute-optimal allocation** of model size and training duration.

The compute-optimal frontier: Kaplan *et al.* found that *larger models are significantly more sample-efficient* – they can reach a given loss with much fewer training examples than smaller models [8]. Therefore, with a fixed compute budget, the optimal strategy is to use a *very large model* and train it on a **relatively modest amount of data, stopping well before full convergence** [8]. In other words, it's better to train a bigger model for fewer steps than a smaller model to completion. This was a somewhat counterintuitive result at the time. Mathematically, for a 10× increase in available compute, Kaplan's law suggested scaling *model parameters* about 5.5× and *dataset size* only about 1.8× [9]. In effect, *model size* was emphasized more than *training duration* or data volume under their optimal scaling rule. This guidance – *"scale model width/depth aggressively, and don't necessarily use all your data"* – influenced the design of models like **GPT-3**, which had 175B parameters but was trained on ~300 billion tokens (stopping short of using any more data to avoid over-training given the budget).

However, an important caveat emerged: Kaplan's analysis didn't account for an absolute lower bound on loss (the "irreducible" loss from data entropy) [10]. Their power laws had no explicit floor, which means they could overestimate the benefits of scaling when loss approaches that floor. By 2022, as models grew much larger (multi-hundred-billion parameters), researchers began to notice discrepancies from the 2020 predictions. This brings us to the *next* major development in scaling laws.

Hoffmann et al. (2022): Revisiting Compute-Optimal Training (the Chinchilla Paper)

Hoffmann *et al.* (2022) – often known by the nickname "*Chinchilla*" paper – revisited scaling laws using new data and larger models. They made a pivotal discovery: *"current large language models are significantly*

First Two Reading Assignments (long things)

Leopold Aschenbrenner

SITUATIONAL AWARENESS

The Decade Ahead

JUNE 2024

arXiv:2108.07258v2 [cs.LG] 18 Aug 2021

On the Opportunities and Risks of Foundation Models

Rishi Bommasani* Drew A. Hudson Ehsan Adeli Russ Altman Simran Arora
Sydney von Arx Michael S. Bernstein Jeannette Bohg Antoine Bosselut Emma Brunskill
Erik Brynjolfsson Shyamal Buch Dallas Card Rodrigo Castellon Niladri Chatterji
Annie Chen Kathleen Creel Jared Quincy Davis Dorottya Demszky Chris Donahue
Moussa Doumbouya Esin Durmus Stefano Ermon John Etchemendy Kawin Ethayarajh
Li Fei-Fei Chelsea Finn Trevor Gale Lauren Gillespie Karan Goel Noah Goodman
Shelby Grossman Neel Guha Tatsunori Hashimoto Peter Henderson John Hewitt
Daniel E. Ho Jenny Hong Kyle Hsu Jing Huang Thomas Icard Saahil Jain
Dan Jurafsky Pratyusha Kalluri Siddharth Karamcheti Geoff Keeling Fereshte Khani
Omar Khattab Pang Wei Koh Mark Krass Ranjay Krishna Rohith Kuditipudi
Ananya Kumar Faisal Ladhak Mina Lee Tony Lee Jure Leskovec Isabelle Levent
Xiang Lisa Li Xuechen Li Tengyu Ma Ali Malik Christopher D. Manning
Suvir Mirchandani Eric Mitchell Zanele Munyikwa Suraj Nair Avanika Narayan
Deepak Narayanan Ben Newman Allen Nie Juan Carlos Niebles Hamed Nilforoshan
Julian Nyarko Giray Ogut Laurel Orr Isabel Papadimitriou Joon Sung Park Chris Piech
Eva Portelance Christopher Potts Aditi Raghunathan Rob Reich Hongyu Ren
Frieda Rong Yusuf Roohani Camilo Ruiz Jack Ryan Christopher Ré Dorsa Sadigh
Shiori Sagawa Keshav Santhanam Andy Shih Krishnan Srinivasan Alex Tamkin
Rohan Taori Armin W. Thomas Florian Tramèr Rose E. Wang William Wang Bohan Wu
Jiajun Wu Yuhuai Wu Sang Michael Xie Michihiro Yasunaga Jiaxuan You Matei Zaharia
Michael Zhang Tianyi Zhang Xikun Zhang Yuhui Zhang Lucia Zheng Kaitlyn Zhou
Percy Liang*[1]

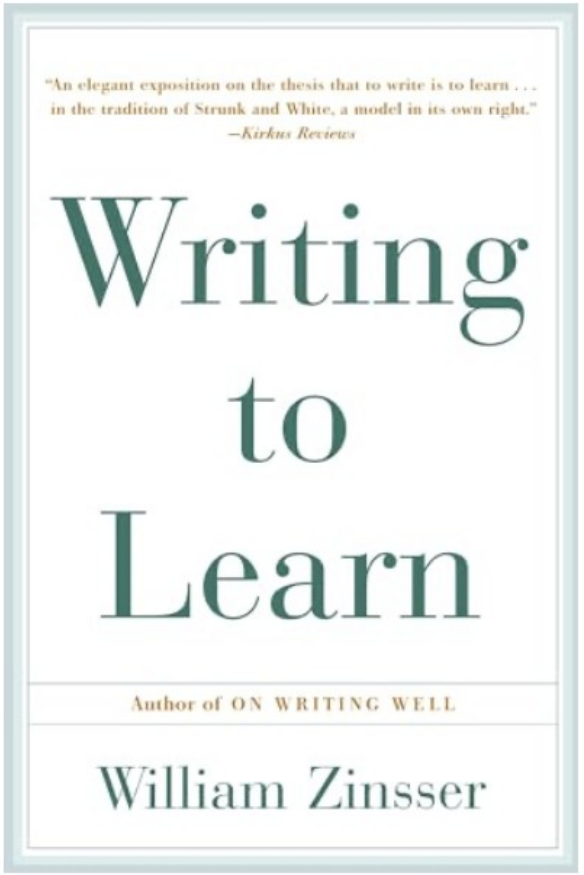
Center for Research on Foundation Models (CRFM)
Stanford Institute for Human-Centered Artificial Intelligence (HAI)
Stanford University

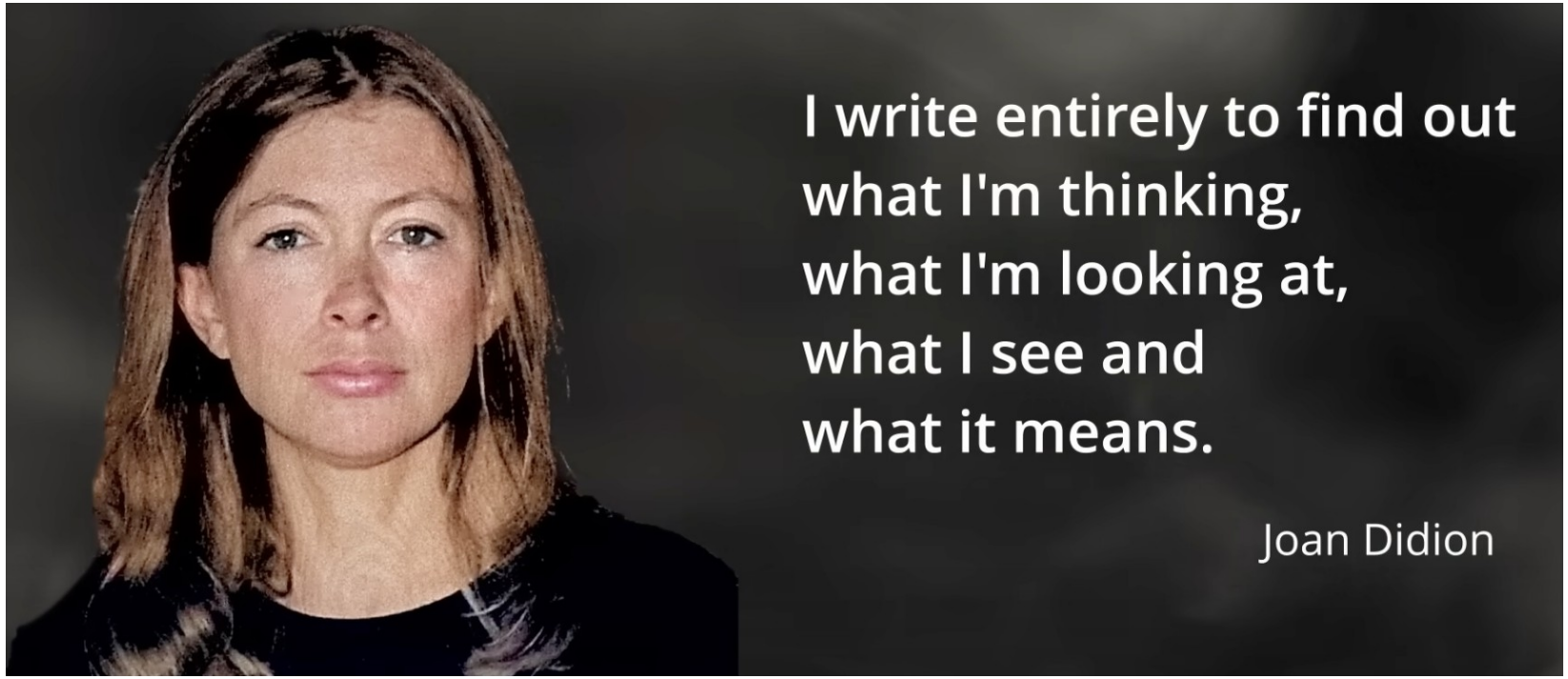
AI is undergoing a paradigm shift with the rise of models (e.g., BERT, DALL-E, GPT-3) that are trained on broad data at scale and are adaptable to a wide range of downstream tasks. We call these models foundation models *to underscore their critically central yet incomplete character. This report provides a thorough account of the opportunities and risks of foundation models, ranging from their capabilities (e.g., language, vision, robotics, reasoning, human interaction) and technical principles (e.g., model architectures, training procedures, data, systems, security, evaluation, theory) to their applications (e.g., law, healthcare, education) and societal impact (e.g., inequity, misuse, economic and environmental impact, legal and ethical considerations). Though foundation models are based on standard deep learning and transfer learning, their scale results in new emergent capabilities, and their effectiveness across so many tasks incentivizes homogenization. Homogenization provides powerful leverage but demands caution, as the defects of the foundation model are inherited by all the adapted models downstream. Despite the impending widespread deployment of foundation models, we currently lack a clear understanding of how they work, when they fail, and what they are even capable of due to their emergent properties. To tackle these questions, we believe much of the critical research on foundation models will require deep interdisciplinary collaboration commensurate with their fundamentally sociotechnical nature.*

[1]Corresponding author: pliang@cs.stanford.edu

*Equal contribution.

I strongly encourage you to journal this class





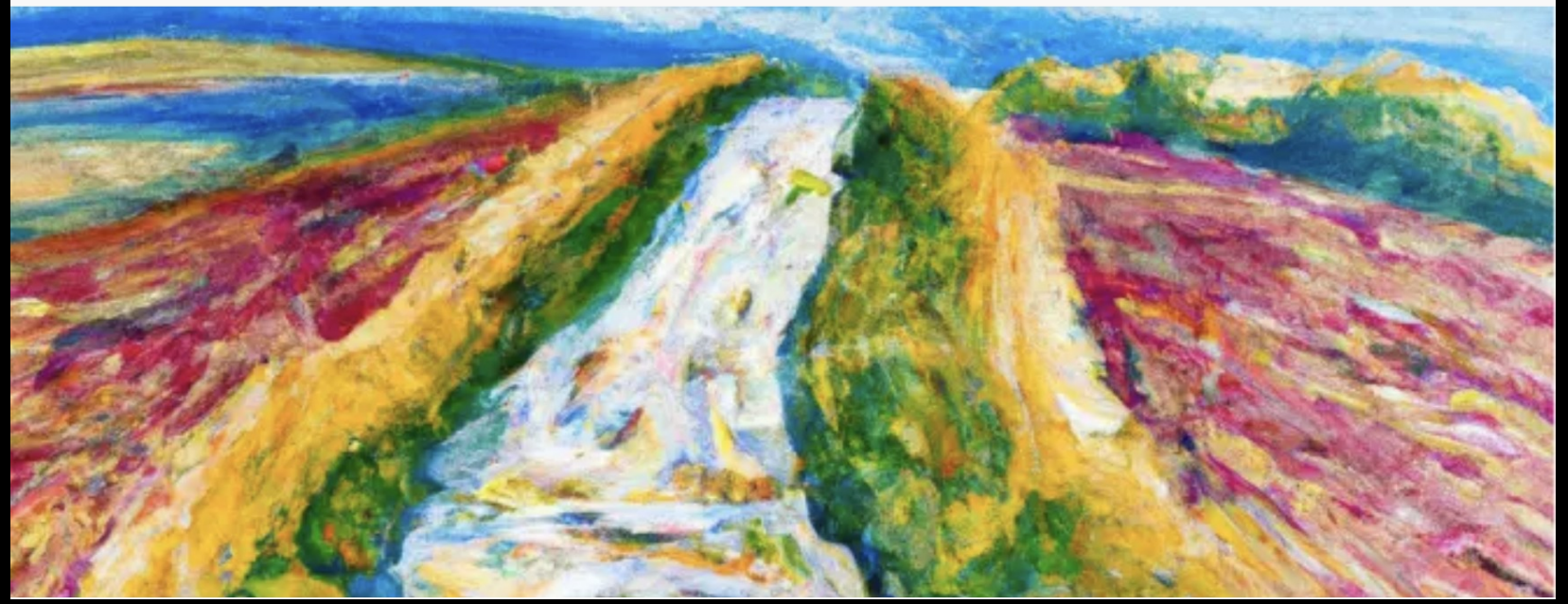
“I wish to God these calculations had been executed by steam.” Charles Babbage, 1822



The Intelligence Age

September 23, 2024

Blog post by Sam Altman



It won't happen all at once, but we'll soon be able to work with AI that helps us accomplish much more than we ever could without AI; eventually we can each have a personal AI team, full of virtual experts in different areas, working together to create almost anything we can imagine. Our children will have virtual tutors who can provide personalized instruction in any subject, in any language, and at whatever pace they need. We can imagine similar ideas for better healthcare, the ability to create any kind of software someone can imagine, and much more.

With these new abilities, we can have shared prosperity to a degree that seems unimaginable today; in the future, everyone's lives can be better than anyone's life is now. Prosperity alone doesn't necessarily make people happy – there are plenty of miserable rich people – but it would meaningfully improve the lives of people around the world.

Here is one narrow way to look at human history: after thousands of years of compounding scientific discovery and technological progress, we have figured out how to melt sand, add some impurities, arrange it with astonishing precision at extraordinarily tiny scale into computer chips, run energy through it, and end up with systems capable of creating increasingly capable artificial intelligence.

This may turn out to be the most consequential fact about all of history so far. It is possible that we will have superintelligence in a few thousand days (!); it may take longer, but I'm confident

How did we get to the doorstep of the next leap in prosperity?
In three words: **deep learning worked**.
In 15 words: **deep learning worked, got predictably better with scale, and we dedicated increasing resources to it.**

That's really it; humanity discovered an algorithm that could really, truly learn any distribution of data (or really, the underlying "rules" that produce any distribution of data). To a shocking degree of precision, the more compute and data available, the better it gets at helping people solve hard problems. I find that no matter

super ⟹ intelligence

- having greater influence, capacity, etc. than another of its kind
- the ability to acquire and apply knowledge and skills

An AI system that is superior to humans in intellectual capabilities and superior to other lesser AIs

An AI system that is superior to the combined intellectual capability of all humans

The AI system that rules them all

An aspirational goal that will drive AI research and development for many decades?

Superintelligence
the idea that eats smart people

--**Maciej Cegłowski**

Ten Scientific Open Questions That Could Improve Life on Earth for All Humans

1. **Achieving Practical Fusion Energy**
   - What are the optimal conditions for sustained and controlled nuclear fusion?
   - How can plasma instabilities be minimized or controlled?
   - What materials can endure the harsh environment inside a fusion reactor?
2. **Curing Major Diseases Like Cancer and Alzheimer's**
   - What genetic and molecular factors initiate and drive these diseases?
   - How do misfolded proteins contribute to disease progression?
   - What compounds can intervene without harmful side effects?
3. **Effective Solutions to Climate Change**
4. **Developing Sustainable Agriculture to Feed the Growing Population**
   - How can we enhance photosynthesis and nutrient utilization in crops?
   - What farming practices maximize yield while preserving the environment?
   - How can we reduce post-harvest losses globally?
5. **Providing Clean Water for All**
6. **Understanding and Reversing Aging**
   - What biological processes are responsible for aging?
   - How can cellular repair mechanisms be enhanced?
   - What are the implications of altering natural aging processes?
7. **Creating Universal Immunity to Viruses**
8. **Developing Efficient Energy Storage Solutions**
9. **Solving the Antibiotic Resistance Crisis**
10. **Removing Pollutants from the Environment**



Thought for 17 seconds

An advanced superintelligence, with unparalleled computational capabilities and a deep understanding of complex systems, could use pure thought to tackle these grand challenges. By leveraging theoretical models, simulations, and comprehensive data analysis, it could uncover novel solutions without the need for physical experimentation. Here's how such an AI might approach each problem and the underlying questions it would need to address:

The Ramp

The Gigawatt Era: From Hyperscale To Exascale

Phillip Marangella Forbes Councils Member

Forbes Technology Council COUNCIL POST | Membership (Fee-Based)



Sam Altman
@sama

we believe the world needs more ai infrastructure--fab
datacenters, etc--than people are currently pla

building massive-scale ai infra
crucial to economic

opp

The CEO of OpenAI is in talks with investors to raise between $5-7 trillion to reshape the global chip industry and thereby ensure the supply of the computing power required for artificial intelligence developments.

1.6K 14K 1.3K

U.S. DEPARTMENT OF ENERGY

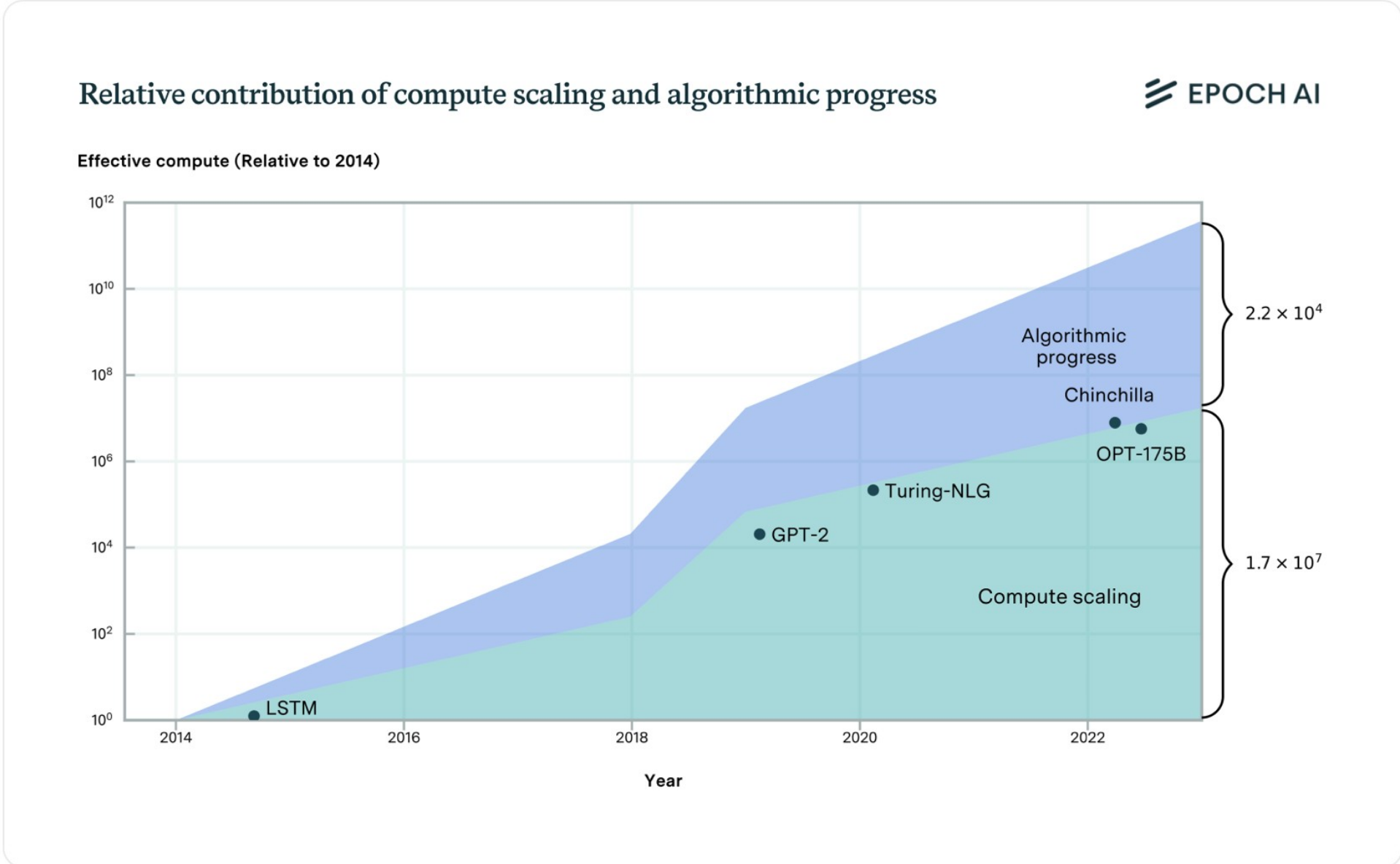


Figure 3: Estimates of the contributions of scaling and algorithmic innovation in terms of the raw compute that would be naively needed to achieve a state-of-the-art level of performance. The contribution of algorithmic progress is roughly half as much as that of compute scaling.

Progress in language model performance surpasses what we'd expect from merely increasing computing resources, occurring at a pace equivalent to doubling computational power every 5 to 14 months.